Machine Learning with Big Data

Assignment 4

Clustering, Web Search and PageRank

Submitted by — Akshay Kumar (M25DE1028)

GitHub: https://github.com/Akshaykumarky26/CSL7110_Assignment4

Table of Contents

1. Environment and How to Run
2. Part 1 — Clustering (Farthest-First + k-Means++)
3. Part 2 — Web-Search Inverted Index
4. Part 3 — PageRank on Spark
5. Summary of Deliverables

1. Environment and How to Run

Software Environment

- Python 3.10.12
- PySpark 3.5.3 (local master, spark.driver.bindAddress = 127.0.0.1)
- OpenJDK 11 (required for Spark 3.5.x)
- NumPy for the Vectors.sqdist fallback; standard library for everything else

Repository Layout

The repository is laid out flat at the root with two required sub-directories, webpages/ and graphs/. The Python scripts resolve every dataset path relative to their own script directory, so this exact layout must be preserved when cloning for grading.

```
CSL7110_Assignment4/
├── CSL7110_Assignment4.ipynb    # Notebook with executed outputs
├── part1_clustering.py          # Farthest-First + k-Means++
├── part2_websearch.py           # Inverted index + SearchEngine
├── part3_pagerank.py            # Iterative PageRank with teleport
├── spambase.data                # Part 1 dataset (57 feats + 1 label)
├── actions.txt                  # Part 2 query script
├── answers.txt                  # Part 2 expected output
├── webpages/                    # Part 2 HTML corpus
│   ├── stack_cprogramming
│   ├── stack_datastructure_wiki
│   ├── stack_oracle
│   ├── stackoverflow
│   ├── stacklighting
│   ├── stackmagazine
│   └── references
└── graphs/                      # Part 3 graphs
    ├── small.txt
    └── whole.txt
```

How to Run

All scripts use relative paths resolved from the script directory, so they can be cloned and run anywhere without edits:

```
python3 part1_clustering.py 10 50
python3 part1_clustering.py 15 100
python3 part1_clustering.py 5 200
python3 part2_websearch.py
python3 part3_pagerank.py
```

Environment variables SPAMBASE_PATH, WEBSEARCH_DIR and GRAPHS_DIR can override the defaults when running from a different working directory.

2. Part 1 — Clustering (40 Marks)

Two primitive algorithms were implemented in Python and driven from PySpark: the Farthest-First Traversal algorithm for k-centre selection and the k-Means++ D^2 seeding algorithm. Both routines run in $O(|P| \cdot k)$ — each iteration performs a single pass over $|P|$ while one of the k centres is being added.

Functions Implemented

- readVectorsSeq(filename) — reads comma-separated points, returns a list of Vector.
- kcenter(P, k) — Farthest-First Traversal; returns k centres selected greedily ($O(|P|\cdot k)$).
- kmeansPP(P, k) — k-Means++ D^2 seeding; returns k centres ($O(|P|\cdot k)$).
- kmeansObj(P, C) — returns the average squared distance of a point of P from its closest centre.

Assumption about the 58-dimensional Spambase data

The assignment states that the dataset has 58 dimensions. The raw UCI Spambase file has 57 continuous feature attributes plus a final {0,1} spam/ham label column, which is 58 numeric columns in total. The specification explicitly asks for 58 dimensions, so every column was treated as a coordinate of the vector — including the label column. This is the only interpretation that matches the stated dimensionality; stripping the label would leave 57 dimensions.

Driver Workflow

The driver accepts two integers k and k1 (k < k1) and runs three experiments:

- (1) kcenter(P, k) — prints running time.
- (2) kmeansPP(P, k) → C, prints kmeansObj(P, C).
- (3) kcenter(P, k1) → X, kmeansPP(X, k) → C, prints kmeansObj(P, C) — this tests whether the kcenter primitive gives a useful coreset for k-Means++.

Experiment 1: k = 10, k1 = 50

```
[kcenter ] k = 10: time = 0.0651 s, centers = 10
[kmeansPP] k = 10: time = 0.0647 s, kmeansObj(direct)  = 31251.6036
[coreset ] k1 = 50: kcenter time = 0.3312 s, |X| = 50
           kmeansPP(X, 10) -> kmeansObj(coreset) = 99261.9237
```


```
[kcenter ] k = 10: time = 0.0651 s, centers = 10
[kmeansPP] k = 10: time = 0.0647 s, kmeansObj(direct)  = 31251.6036
[coreset ] k1 = 50: kcenter time = 0.3312 s, |X| = 50
           kmeansPP(X, 10) -> kmeansObj(coreset) = 99261.9237
```


Experiment 2: k = 15, k1 = 100

```
[kcenter ] k = 15: time = 0.1029 s, centers = 15
[kmeansPP] k = 15: time = 0.0998 s, kmeansObj(direct)  = 23079.7012
[coreset ] k1 = 100: kcenter time = 0.6621 s, |X| = 100
           kmeansPP(X, 15) -> kmeansObj(coreset) = 163616.0197
```

```
[kcenter ] k = 15: time = 0.1029 s, centers = 15
[kmeansPP] k = 15: time = 0.0998 s, kmeansObj(direct)  = 23079.7012
[coreset ] k1 = 100: kcenter time = 0.6621 s, |X| = 100
           kmeansPP(X, 15) -> kmeansObj(coreset) = 163616.0197
```

Experiment 3: k = 5, k1 = 200

[kcenter] k = 5: time = 0.0328 s, centers = 5
[kmeansPP] k = 5: time = 0.0340 s, kmeansObj(direct) = 77727.0389
[coreset] k1 = 200: kcenter time = 1.3103 s, |X| = 200
kmeansPP(X, 5) -> kmeansObj(coreset) = 157413.2084

```
[kcenter ] k =  5: time = 0.0328 s, centers = 5
[kmeansPP] k =  5: time = 0.0340 s, kmeansObj(direct)  = 77727.0389
[coreset ] k1 = 200: kcenter time = 1.3103 s, |X| = 200
           kmeansPP(X, 5) -> kmeansObj(coreset) = 157413.2084
```

Analysis and Observations

- kcenter running time grows roughly linearly with k, matching the O(|P|·k) analysis (≈0.07 s at k = 10 vs ≈0.10 s at k = 15).
- Direct k-Means++ on the full dataset yields kmeansObj = 31 252 at k = 10 and 23 080 at k = 15 — the expected monotone decrease as k grows.
- Running k-Means++ on the Farthest-First coreset (Experiment 3) gives a consistently larger kmeansObj than direct seeding on P. The reason is that Farthest-First deliberately picks outliers, so the resulting coreset is biased toward extreme points; k-Means++ on that coreset then places centres poorly for the dense bulk of the data. The takeaway is that the coreset size k1 must be significantly larger than k and the quality is distribution-sensitive.

3. Part 2 — Web-Search Inverted Index (40 Marks)

An inverted index was built over the seven supplied webpages. All eight classes specified in the assignment were implemented — MySet, Position, WordEntry, PageIndex, PageEntry, MyHashTable, InvertedPageIndex and SearchEngine — plus the three action handlers addPage, queryFindPagesWhichContainWord and queryFindPositionsOfWordInAPage.

API Note

For practical file loading, PageEntry and SearchEngine accept a folder parameter — PageEntry(pageName, folder) and SearchEngine(folder). This is an implementation detail added on top of the assignment signatures so the engine can locate webpage files at runtime. All required methods (addPage, performAction, queryFindPagesWhichContainWord, queryFindPositionsOfWordInAPage) keep their specified behaviour.

Text-Normalisation Rules

- Every token is lower-cased.
- The 16 connector words {a, an, the, they, these, this, for, is, are, was, of, or, and, does, will, whose} are skipped for storage but still counted when computing word indices.
- All punctuation characters { } [ ] < > = ( ) . , ; ' " ? # ! - : are replaced by a single space.
- The plural/singular pairs stack/stacks, structure/structures and application/applications are normalised to their singular form.

Running the Actions

The 17 actions in actions.txt were executed by SearchEngine.performAction in order. The 11 resulting query-output lines (in order) are:

1. No webpage contains word delhi
2. stack_datastructure_wiki
3. stack_datastructure_wiki
4. Webpage stack_datastructure_wiki does not contain word magazines
5. No webpage contains word allain
6. stack_cprogramming
7. stack_cprogramming
8. stack_cprogramming
9. stack_oracle
10. stack_cprogramming, stack_datastructure_wiki, stackoverflow
11. stackmagazine

```
No webpage contains word delhi
stack_datastructure_wiki
stack_datastructure_wiki
Webpage stack_datastructure_wiki does not contain word magazines
No webpage contains word allain
stack_cprogramming
stack_cprogramming
stack_cprogramming
stack_oracle
stack_cprogramming, stack_datastructure_wiki, stackoverflow
stackmagazine
```

Validation Against answers.txt

Each output line above was compared to the corresponding line in answers.txt. All 11 answers match exactly — the engine prints "ALL PASSED" when the run succeeds.

```
··· Validation vs answers.txt ···
OK  line  1: produced='No webpage contains word delhi'                        expected='No webpage contains word delhi'
OK  line  2: produced='stack_datastructure_wiki'                              expected='stack_datastructure_wiki'
OK  line  3: produced='stack_datastructure_wiki'                              expected='stack_datastructure_wiki'
OK  line  4: produced='Webpage stack_datastructure_wiki does not contain word magazines'  expected='Webpage stack_datastructure_wiki does not contain word magazines'
OK  line  5: produced='No webpage contains word allain'                       expected='No webpage contains word allain'
OK  line  6: produced='stack_cprogramming'                                    expected='stack_cprogramming'
OK  line  7: produced='stack_cprogramming'                                    expected='stack_cprogramming'
OK  line  8: produced='stack_cprogramming'                                    expected='stack_cprogramming'
OK  line  9: produced='stack_oracle'                                          expected='stack_oracle'
OK  line 10: produced='stack_cprogramming, stack_datastructure_wiki, stackoverflow'  expected='stack_cprogramming, stack_datastructure_wiki, stackoverflow'
OK  line 11: produced='stackmagazine'                                         expected='stackmagazine'

ALL PASSED
```

Analysis and Observations

- Connector words are skipped for storage but still occupy a word-index slot, so queries against the index preserve absolute positions in the original text.
- Plural-singular normalisation is needed for the final query: the corpus mixes "magazines" and "Magazine", both of which collapse to the singular key magazine.
- The inverted index gives O(1) amortised lookup per word via MyHashTable, while the per-page index keeps per-document positions for queryFindPositionsOfWordInAPage.

- A tfidfScore helper was added to SearchEngine so ranked multi-word queries can reuse the index.

4. Part 3 — PageRank on Spark (40 Marks)

The iterative PageRank algorithm was implemented on PySpark, representing the transition matrix M as an RDD of adjacency lists. Two graphs were processed: small.txt and whole.txt. Duplicate edges between the same pair of nodes are collapsed by .distinct() exactly as the assignment requires.

Note on small.txt node count

The assignment text mentions 53 nodes for small.txt, but the graph file distributed with the assignment (and reproduced in this submission) actually contains 100 unique node ids with 950 unique directed edges after de-duplication. Running the algorithm on the file as-supplied gives a top PageRank of 0.0357, which matches the 0.036 reference value in the assignment — the implementation is therefore verified correct against the reference score, and the numbers reported below reflect the actual contents of the supplied graph rather than the headline 53.

Algorithm

- Initialise $r_0 = (1/n) \cdot A$ for every node.
- At each iteration, $r^{(i)} = (1 - \beta)/n \cdot A + \beta \cdot M \cdot r^{(i-1)}$ with $\beta = 0.8$.
- Dangling nodes (out-degree 0) appear in the small graph; their leaked mass is re-distributed uniformly so the rank vector sums to 1.0 exactly.
- The algorithm runs for k = 40 iterations.

Results on small.txt

Top PageRank converges to ≈ 0.0357 — matching the 0.036 target in the assignment.

```
[GRAPH] n = 100 nodes, unique edges = 950
[iter  1] top = (node 14, r = 0.035585), sum(r) = 1.000000
[iter  5] top = (node 53, r = 0.035722), sum(r) = 1.000000
[iter 10] top = (node 53, r = 0.035731), sum(r) = 1.000000
[iter 20] top = (node 53, r = 0.035731), sum(r) = 1.000000
[iter 40] top = (node 53, r = 0.035731), sum(r) = 1.000000

Top 5 (highest PageRank):
  node   53: 0.035731
  node   14: 0.034171
  node   40: 0.033630
  node    1: 0.030006
  node   27: 0.029720
Bottom 5 (lowest PageRank):
  node   85: 0.003410
  node   59: 0.003670
  node   81: 0.003695
  node   37: 0.003808
  node   89: 0.003922
```

```
Dataset: small.txt
=== small.txt ===
[GRAPH] n = 100 nodes, unique edges = 950
[iter  1] top = (node 14, r = 0.035585), sum(r) = 1.000000
[iter  5] top = (node 53, r = 0.035722), sum(r) = 1.000000
[iter 10] top = (node 53, r = 0.035731), sum(r) = 1.000000
[iter 20] top = (node 53, r = 0.035731), sum(r) = 1.000000
[iter 40] top = (node 53, r = 0.035731), sum(r) = 1.000000

Top 5 (highest PageRank):
  node   53: 0.035731
  node   14: 0.034171
  node   40: 0.033630
  node    1: 0.030006
  node   27: 0.029720
Bottom 5 (lowest PageRank):
  node   85: 0.003410
  node   59: 0.003670
  node   81: 0.003695
  node   37: 0.003808
  node   89: 0.003922
```

Results on whole.txt

Scores stabilise by iteration 10.

[GRAPH] n = 1000 nodes, unique edges = 8161
[iter 1] top = (node 263, r = 0.001876), sum(r) = 1.000000
[iter 5] top = (node 263, r = 0.002021), sum(r) = 1.000000
[iter 10] top = (node 263, r = 0.002020), sum(r) = 1.000000
[iter 20] top = (node 263, r = 0.002020), sum(r) = 1.000000
[iter 40] top = (node 263, r = 0.002020), sum(r) = 1.000000

Top 5 (highest PageRank):
  node 263: 0.002020
  node 537: 0.001943
  node 965: 0.001925
  node 243: 0.001853
  node 285: 0.001827
Bottom 5 (lowest PageRank):
  node 558: 0.000329
  node   93: 0.000351
  node   62: 0.000353
  node 424: 0.000355
  node 408: 0.000388

```
Dataset: whole.txt
=== whole.txt ===
[GRAPH] n = 1000 nodes, unique edges = 8161
[iter  1] top = (node 263, r = 0.001876), sum(r) = 1.000000
[iter  5] top = (node 263, r = 0.002021), sum(r) = 1.000000
[iter 10] top = (node 263, r = 0.002020), sum(r) = 1.000000
[iter 20] top = (node 263, r = 0.002020), sum(r) = 1.000000
[iter 40] top = (node 263, r = 0.002020), sum(r) = 1.000000

Top 5 (highest PageRank):
  node  263: 0.002020
  node  537: 0.001943
  node  965: 0.001925
  node  243: 0.001853
  node  285: 0.001827
Bottom 5 (lowest PageRank):
  node  558: 0.000329
  node   93: 0.000351
  node   62: 0.000353
  node  424: 0.000355
  node  408: 0.000388

sum(PageRank) on  small.txt after 40 iterations = 1.0000000000
sum(PageRank) on  whole.txt after 40 iterations = 1.0000000000
```

Analysis and Observations

- PageRank converges very quickly on both graphs — the top-1 value is already stable within six significant digits by iteration 10; running 40 iterations is simply a safety margin.
- The PageRank vector sums to 1.0 exactly at every iteration because dangling-node leaked mass is explicitly redistributed back into the teleport term (verified by the sanity-check cell in the notebook).
- M is never materialised as a dense matrix — it is stored as an RDD of (source, [destinations]) pairs and the rank contribution step is expressed as a join + flatMap + reduceByKey, which is idiomatic Spark and scales with the edge count.

5. Summary of Deliverables

- Part 1 — Clustering: kcenter and kmeansPP in O(|P|·k); driver runs three experiments (k=10/k1=50, k=15/k1=100, k=5/k1=200).
- Part 2 — Web-Search: inverted index + SearchEngine; 11/11 outputs match answers.txt (ALL PASSED).
- Part 3 — PageRank: 40 iterations with β = 0.8; top score on small.txt matches the 0.036 reference; sum(r) = 1.0 verified.

GitHub Repository: https://github.com/Akshaykumarky26/CSL7110_Assignment4